



UNIVERZITET U NOVOM SADU
FAKULTET TEHNIČKIH NAUKA



PROJEKAT IZ DIGITALNIH SISTEMA OTPORNIH NA GREŠKE

Primena *pair-and-a-spare* tehnike otpornosti na greške na primeru FIR filtra

Asistent : Jana Janković

Student : Nadežda Jevremović E1 92/2024

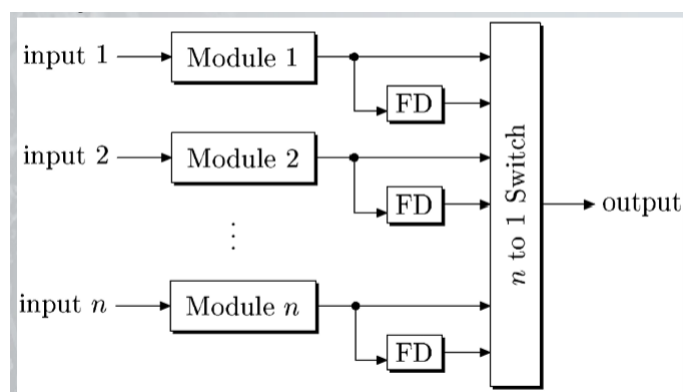
1. Uvod — Pair-and-a-Spare tehnika otpornosti na greške

Pair-and-a-Spare je aktivna tehnika redundanse koja kombinuje dobre osobine dve osnovne tehnike aktivne redundanse: standby sparing tehnike i tehnike duplication with comparison.

1.1 Standby sparing

Standby sparing je jedna od tehnika aktivne redundanse. Samo jedan od n modula je operativan i obezbeđuje izlaz sistema, dok preostalih $n - 1$ modula služi kao rezerva. Rezerva je suvišna komponenta koja nije potrebna za normalan rad sistema u trenutku kada je aktivni modul ispravan.

Prekidač (switch) je uređaj koji prati aktivni modul i prebacuje rad sistema na rezervni modul ukoliko jedinica za detekciju kvara (fault detection unit, FD) prijavi grešku. FD blok se sastoji od modula koji obavlja istu funkcionalnost kao i aktivni modul, i komparatora koji postavlja jedinicu na portu error signal ukoliko dođe do razlike između vrednosti izlaznih podataka ta dva modula.

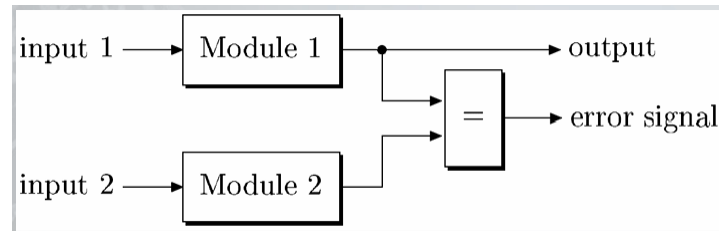


1.2 Duplication with comparison

Aktivna redundansa postiže toleranciju na kvarove tako što najpre otkriva kvar, a zatim izvršava potrebne radnje da bi sistem oporavio i vratio se u operativno stanje. Tehnike aktivne redundanse su česte u aplikacijama gde su privremeno pogrešni rezultati poželjniji od visokog stepena redundanse potrebne za maskiranje kvara — dozvoljene su povremene greške, dokle god se sistem oporavlja i vraća u operativno stanje.

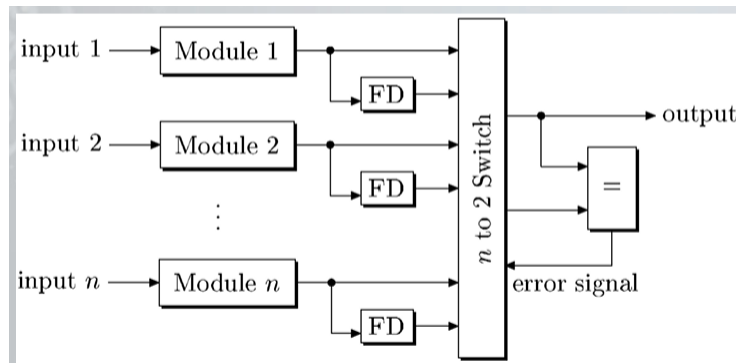
Osnovni oblik aktivne redundanse jeste duplication with comparison. Dva identična modula obavljaju isto računanje paralelno, a rezultati se poređuju pomoću komparatora. Ukoliko se rezultati ne slažu, generiše se signal greške.

Bitno ograničenje ove tehnike: duplication with comparison može da otkrije samo postojanje kvara jednog od dva modula, ali ne može da utvrdi koji je od njih неисправan (jer su oba jednako verodostojna — nema treće referentne tačke za poređenje). Nakon otkrivanja kvara nijedna korektivna akcija se ne preduzima — sistem samo signalizuje da postoji problem, ali se ne oporavlja.



1.3 Pair-and-a-Spare tehnika

Ideja pair-and-a-spare je slična standby sparing tehnici, s tom razlikom što, umesto jednog aktivnog modula, u svakom trenutku dva modula rade paralelno. Kao i u slučaju duplication with comparison, rezultati ta dva modula se porede radi detekcije neslaganja.



Ako se iz komparatora primi signal greške, prekidač (switch logika) koristi taj izveštaj iz bloka za detekciju i odlučuje koji od dva aktivna izlaza je неисправan. Ovde je ključna razlika u odnosu na duplication with comparison: pošto sistem ima dodatne rezervne module, kada se detektuje neslaganje, prekidač ima mogućnost da utvrdi koji je od dva modula неисправan (npr. daljim poređenjem sa trećim modulom) i da ga ukloni i zameni rezervnim modulom koji nije u kvaru. Time se sistem, za razliku od duplication with comparison, aktivno oporavlja i vraća u ispravno operativno stanje, umesto da samo signalizuje grešku.

Pair-and-a-spare tako spaja prednosti obe tehnike: manju potrošnju energije u odnosu na duplication with comparison (jer nisu svi moduli sistema aktivni istovremeno, već samo dva u svakom trenutku, dok ostali čekaju u standby režimu), i sposobnost lokalizacije i otklanjanja kvara koju standby sparing ima, ali sa pouzdanijom detekcijom greške jer se poređenje uvek vrši između dva trenutno aktivna modula.

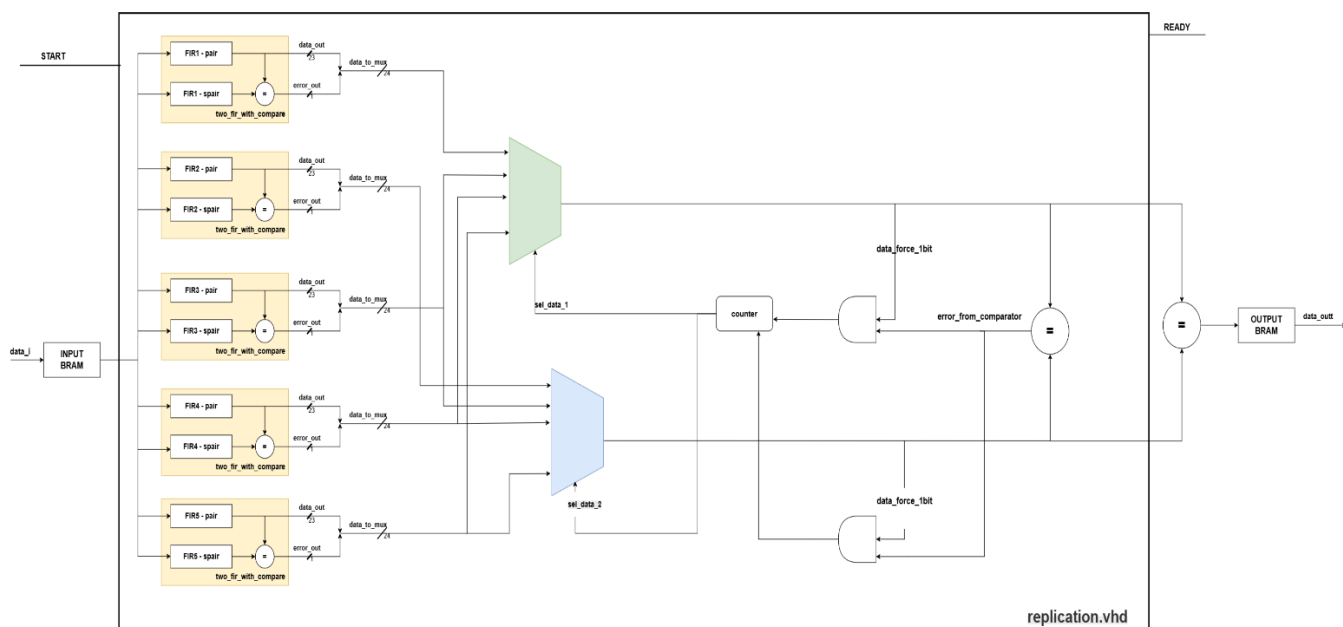
1.4 Broj tolerisanih grešaka i granica ispravnog rada

Sistem realizovan pair-and-a-spare tehnikom sa n modula toleriše maksimalno $n - 1$ grešaka. Ovo proizlazi iz principa rada tehnike: u svakom trenutku su dva modula aktivna i upoređuju se, a preostalih $n - 2$ su u rezervi. Svaki put kada se detektuje kvar, neispravan modul se isključuje i zamenjuje jednim od rezervnih, sve dok rezervni moduli ne budu iskorišćeni.

Važno je precizirati šta se dešava nakon što se iscrpe svi rezervni moduli, jer se ovo često pogrešno formuliše. Kada se desi n -ta greška (odnosno kada se potroši i poslednji rezervni modul), sistem se svodi na konfiguraciju u kojoj postoji samo jedan preostali ispravan modul — a pošto poređenje zahteva dva aktivna modula da bi se greška uopšte detektovala, ta poslednja greška se ne može više detektovati. Sistem u tom trenutku ne prestaje fizički da radi, ali gubi sposobnost samoprovere: nastavlja da generiše izlaz, ali bez garancije da je taj izlaz ispravan, jer nema više mehanizma poređenja koji bi to potvrdio.

Pair-and-a-spare sa n modula garantuje ispravan rad i detekciju grešaka za $n - 1$ kvarova, dok se za n -ti kvar gubi garancija ispravnosti sistema u smislu mogućnosti detekcije, čak i ako izlazni signal formalno i dalje postoji.

2. Realizacija Pair-and-a-Spare arhitekture na primeru FIR filtra



Realizovani sistem primenjuje pair-and-a-spare tehniku iz poglavlja 1 na konkretnom primeru FIR (Finite Impulse Response) filtra. Umesto jednog FIR modula, dizajn instancira `number_of_replication` identičnih FIR replika, od kojih su u svakom trenutku dve aktivne i upoređuju se, dok ostale čekaju u rezervi, tačno prema principu opisanom u poglavlju 1.3.

Modul top obezbeđuje ulazno/izlazno smeštanje podataka (BRAM) i upravljanje tokom obrade, replication implementira samu pair-and-a-spare logiku (MUX-eve, brojač grešaka, prekidačku logiku), a `two_fir_with_compare` predstavlja jedan FD (fault detection) blok — modul čiji je zadatak da interno udvostruči FIR filter i sam prijavi grešku, koristeći princip iz poglavlja 1.1 na nivou pojedinačne replike.

2.1 Ulazni i izlazni memorijski blokovi (bram, deo top.vhd)

Modul bram je jednostavna sinhrona memorija sa odvojenim adresama za čitanje i upisivanje (`addr_read`, `addr_write`), parametrizovana širinom reči (`RAM_WIDTH`) i dubinom (`RAM_DEPTH`, podrazumevano 4096 odbiraka, `ADDR_SIZE` = 12 bita za adresiranje). U `top.vhd` se dve instance ovog modula koriste kao:

- **ulazni BRAM** — čuva ulazne odbirke signala koji se filtrira; adresa za čitanje (`address_input_bram`) se automatski inkrementira svakim taktom nakon što se start postavi na jedinicu, sve dok se ne pročita svih `RAM_DEPTH` odbiraka;

- **izlazni BRAM** — prima filtrirane odbirke iz pair-and-a-spare bloka (data_to_output_bram) i upisuje ih na adresu koja se takođe inkrementira sinhrono sa start signalom (address_output_bram).

Signal ready se generiše kada adresa upisa u izlazni BRAM dostigne poslednju validnu vrednost (RAM_DEPTH - 1), čime se spoljnom sistemu (npr. testbench-u ili nadređenom kontroleru) signalizira da je obrada svih odbiraka završena i da se rezultati mogu bezbedno očitati sa data_out porta.

2.2 Fault detection blok — two_fir_with_compare (dopunjen opis)

Modul two_fir_with_compare instancira dva potpuno identična fir_param filtra (first_module, second_module), koja u paralelno obrađuju isti ulazni podatak (data_in) sa istim koeficijentima — čist primer duplication with comparison principa iz poglavlja 1.2, primenjen na nivou jedne pair-and-a-spare replike.

Poređenje i signal greške. Komparator (proces error_detection) upoređuje izlaze oba interna FIR filtra (first_data_o_s, second_data_o_s) na svaku uzlaznu ivicu takta:

```
if (first_data_o_s /= second_data_o_s) then
```

```
    error_s <= '1';
```

```
end if;
```

Bitna projektantska karakteristika: ovaj proces nema else granu, pa se error_s ponaša kao *latch* (zapamćena greška) — jednom postavljen na 1, signal error_out ostaje 1 do kraja rada sistema, bez obzira na to što bi se izlazi dva interna filtra kasnije mogli ponovo poklopiti (npr. ako je greška bila prolazna, izazvana jednokratnim fault injection-om). Ovo je namerno drugačije ponašanje od poređenja na nivou replication.vhd (poglavlje 2.4), gde se error_from_comparator ažurira svaki takt i vraća na 0 čim se izlazi aktivnih modula ponovo poklope. Praktična posledica je da jednom detektovana greška unutar pojedinačne replike trajno diskvalifikuje tu repliku iz daljeg korišćenja, čak i ako je uzrok greške bio privremen — što je u skladu sa fizičkom intuicijom da hardverski kvar (npr. stuck-at greška simulirana preko add_force) po pravilu nije prolazan.

Izlazni podatak replike. Iako oba interna FIR filtra rade paralelno, na izlaz modula (data_out) se prosleđuje isključivo rezultat prvog filtra (first_data_o_s):

```
if we_in = '1' then
```

```
    data_out_s <= first_data_o_s;
```

```
else
```

```
    data_out_s <= (others => '0');
```

```
end if;
```

Drugi interni filter (`second_module`) time ima isključivo ulogu provere (FD funkcije), a ne alternativnog izvora podatka — njegov izlaz se nikada ne prosleđuje dalje kao koristan rezultat, već služi samo kao referentna vrednost za komparator. Dodatno, izlaz je uslovljen signalom `we_in`: sve dok se koeficijenti filtra upisuju (`we_in = '1'`), na izlazu se pojavljuje `first_data_o_s`, dok se u suprotnom (`we_in = '0'`) izlaz prisilno postavlja na nulu. Ovim se sprečava da se na izlaz replike neplanirano provuku netačni ili tranzijentni rezultati u fazi pre nego što su svi koeficijenti filtra ispravno učitani.

`dont_touch` atributi. Kao i u `replication.vhd`, signali `first_data_o_s` i `second_data_o_s` označeni su atributom `dont_touch`, čime se sprečava da Vivado sinteza, prepoznavši da oba `fir_param` modula imaju identične ulaze i strukturu, izvrši optimizaciju koja bi ih spojila u jednu logičku granu (`resource sharing`) ili potpuno uklonila drugi modul kao "redundantan" u smislu logičke funkcije. Bez ovog atributa, sinteza bi mogla eliminisati samu suštinu `duplication with comparison` pristupa, jer bi po čisto funkcionalnoj logici (isti ulazi, ista funkcija $\Rightarrow$ isti izlaz) drugi filter delovao nepotreban — što je tačno sve dok ne dođe do fizičke greške koju simulacija `fault injection`-a treba da otkrije.

2.3 Osnovna filterarska jedinica — `fir_param` i `mac`

`fir_param` implementuje direktnu formu FIR filtra reda `fir_ord` (podrazumevano 5, odnosno 6 koeficijenata) kao lanac od `fir_ord + 1` `mac` (`multiply-accumulate`) sekcija, povezanih sistolički: izlaz jedne MAC sekcije (`sec_o`) je ulaz sledeće (`sec_i`).

Koeficijenti filtra (`b_s`) se upisuju sinhrono preko para signala `we_i / coef_addr_i / coef_i`, pri čemu prva sekcija (`first_section`) koristi poslednji koeficijent po indeksu (`b_s(fir_ord)`), a svaka naredna sekcija koristi koeficijente u opadajućem redosledu (`b_s(fir_ord - i)`). Ovim redosledom je efektivno definisan smer prolaska odbirka kroz filterarsku strukturu.

Svaka `mac` instanca u jednom taktu:

1. registruje ulazni odbirak (`u_i`) i pripadajući koeficijent (`b_i`) — `reg_in1`, `reg_in2`;
2. množi ih kao potpisane (`signed`) 24-bitne brojeve, dobijajući 48-bitni proizvod;
3. sabira taj proizvod sa akumuliranom vrednošću iz prethodne sekcije (`sec_i`), i rezultat upisuje u 48-bitni registar `reg_s`, koji predstavlja `sec_o`.

Atribut `use_dsp = "yes"` eksplicitno usmerava sintezu da za operaciju množenja-i-sabiranja koristi namenske DSP48 blokove FPGA čipa, umesto generičke logike.

Konačan izlaz `fir_param` modula (`data_o`) dobija se izdvajanjem bitova (46 downto 23) iz poslednjeg 48-bitnog akumulatora (`mac_inter(fir_ord)`), čime se format rezultata vraća na ulaznu širinu od 24 bita. Ovo je direktno sečenje bitskog vektora, bez zaokruživanja i bez zaštite

od prekoračenja (saturacije) — ukoliko akumulacija probije opseg 48-bitnog potpisanog broja, rezultat se "umotava" (wraparound), a ne saturira.

2.4 Pair-and-a-Spare logika — replication.vhd

Modul replication je centralni deo realizacije i direktno implementira tehniku opisanu u poglavlju 1.3. Sadrži number_of_replication instanci two_fir_with_compare (generate petlja replication_of_fir), čiji se izlazi (filtrirani podatak plus interni error bit) dovode na ulaze dva multipleksera.

Raspodela ulaza multipleksera → Prvi multiplekser (MUX 1) bira između izlaza svih replika osim replike indeksa 1 (tj. bira iz skupa {0, 2, 3, 4, ...}), dok drugi multiplekser (MUX 2) bira između svih replika osim replike indeksa 0 (skup {1, 2, 3, 4, ...}). Ovim rasporedom su u početnom stanju (sel_data_1 = sel_data_2 = 0) aktivni parovi tačno replike 0 i 1 — što odslikava par aktivnih modula iz teorijskog modela pair-and-a-spare tehnike, dok replike 2 do number_of_replication - 1 predstavljaju rezervu.

Detekcija greške → Poređenje se vrši nad izlazima oba multipleksera (data_from_mux_1 i data_from_mux_2), isključivo nad filtriranim podatkom (bitovi output_data_width-1 downto 1), ne i nad internim error bitom pojedinačne replike. Ako se ta dva izlaza ne poklapaju, signal error_from_comparator se postavlja na 1, a signal spremnosti fir_ready na 0 u tom taktu — direktna implementacija komparatorskog bloka iz poglavlja 1.2/1.3.

Prekidačka (switch) logika i zamena rezervom → Kada se detektuje neslaganje (error_from_comparator = '1'), sistem gleda interni error bit (data_from_mux_1(0) odnosno data_from_mux_2(0)) da bi utvrdio koji je od dva trenutno aktivna modula neispravan — jer svaka replika sama prijavljuje sopstvenu grešku preko FD bloka opisanog u 2.2. Ovo je konkretna realizacija mehanizma lokalizacije kvara koji, prema poglavlju 1.3, pair-and-a-spare tehnika ima za razliku od proste duplication with comparison tehnike:

- ako grešku prijavljuje modul selektovan preko sel_data_1, brojač counter se inkrementira i taj selekcionirani signal se pomera na sledeći, još neiskorišćeni rezervni modul;
- ako grešku prijavljuje modul selektovan preko sel_data_2, isti postupak se sprovodi nad sel_data_2.

Time se neispravna replika isključuje iz aktivnog para, a na njeno mesto ulazi sledeća rezervna replika — sistem se aktivno oporavlja, umesto da samo signalizuje grešku.

Izlazna logika → Konačan filtrirani podatak (`data_outt_s`) uzima se od onog od dva aktivna modula koji ne prijavljuje sopstvenu internu grešku (birano preko bita `data_from_mux_1(0)` / `data_from_mux_2(0)`), čime se na izlaz uvek prosleđuje rezultat modula za koji sistem procenjuje da je ispravan.

Iscrpljivanje rezervi → Signal checker je fiksiran na vrednost `number_of_replication` i predstavlja gornju granicu brojača grešaka. Kada counter dostigne vrednost checker — što se prema poglavlju 1.4 dešava tačno nakon $n - 1$ detektovanih i otklonjenih grešaka — sistem više nema preostalih rezervnih modula. U tom slučaju je u kodu eksplicitno predviđeno da se izlaz (`data_outt_s`) prisilno postavi na nulu, umesto da se prosledi rezultat poslednjeg preostalog (i više neproverljivog) modula. Ovo je svesna projektantska odluka koja fizički signalizuje granični slučaj opisan u poglavlju 1.4: sistem ne pokušava da prikaže rezultat čija se ispravnost više ne može potvrditi poređenjem, već umesto toga eksplicitno pokazuje da je izgubio sposobnost samoprovere.

2.5 Napomena o širini signala i "dont_touch" atributima

Izlazna širina prosleđena u replication (`output_data_width` na nivou `top.vhd` iznosi `output_data_width + 1`) namerno je uvećana za jedan bit u odnosu na širinu FIR podatka, kako bi se u okviru istog vektora (`data_to_mux`) nosio i dodatni bit greške (`error_out`) svake replike, bez potrebe za posebnom sabirnicom za error signale. Bit greške je pozicioniran kao najmanje značajan bit (LSB), dok filtrirani podatak zauzima preostalih 24 bita.

Atribut `dont_touch` je postavljen na signale `data_to_mux`, `data_to_mux_1`, `data_to_mux_2`, `sel_data_1` i `sel_data_2` kako bi se sprečilo da Vivado sinteza optimizuje ili ukloni ove signale (npr. `constant propagation` ili `logic sweeping`), što je bitno jer se upravo nad njima kasnije vrši fault injection (opisano u sledećem poglavlju) — da bi injektovana greška bila vidljiva i imala efekat, signal ne sme biti optimizovan iz finalnog netlist-a.

3. Metodologija verifikacije

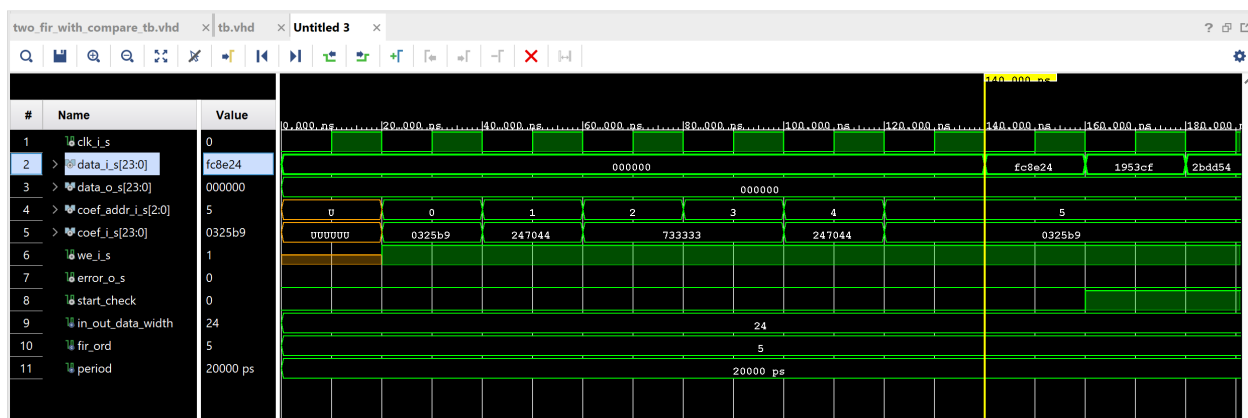
Verifikacija realizovanog sistema sprovedena je kroz tri odvojena testbench-a, od kojih svaki cilja drugi nivo hijerarhije opisane u poglavlju 2. Ovim pristupom se greška, ukoliko postoji, lokalizuje na nivou pojedinačne komponente pre nego što se testira ceo integrisan sistem.

Testbench	Modul pod testom (UUT)	Svrha
tb_two_fir	two_fir_with_compare	Provera FD bloka — da li se dva interna FIR filtra slažu i da li se error_out ispravno generiše
tb_fir_param	fir_param	Provera samog FIR filtra naspram bit-tačnog golden modela (poređenje sa expected.txt)
tb_top	top	Provera celog sistema sa BRAM memorijama i pair-and-a-spare logikom (poređenje sa expected_top.txt)

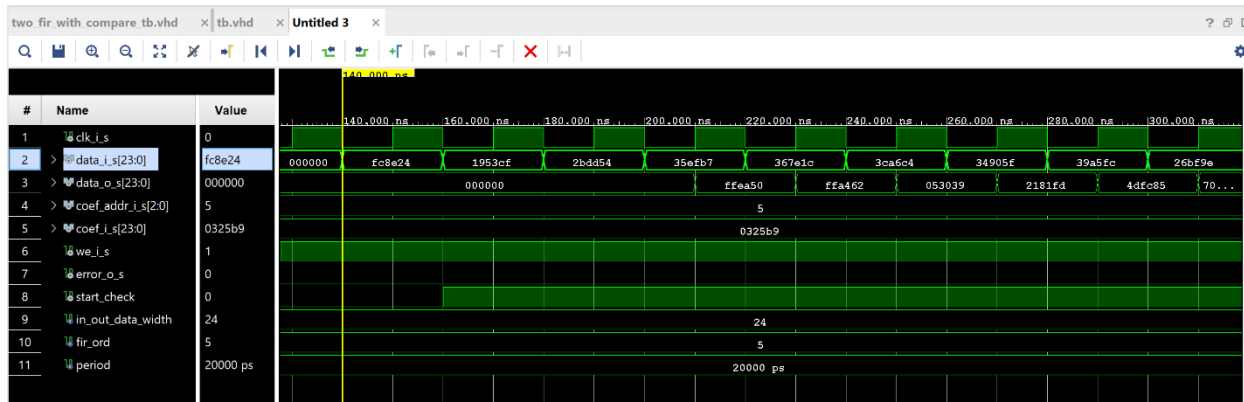
Sva tri testbench-a dele isti osnovni obrazac: generišu takt perioda 20 ns, čitaju koeficijente i ulazne odbirke iz spoljnih .txt fajlova pomoću std.textio biblioteke i pomoćnog paketa txt_util (funkcija to_std_logic_vector), i primenjuju ih na ulaze UUT-a sinhrono sa taktom.

3.1 Testbench nivoa FD bloka (two_fir_with_compare)

Pored upisa koeficijenata i primene ulaznih odbiraka, ova verzija testbench-a sadrži i check_process koji čita golden vrednosti iz expected.txt i poredi ih sa data_o_s, sa tolerancijom ± 7 (isto obrazloženje kao u poglavlju 3.2). Da bi se sinhronizacija uskladila sa stim_process procesom (koji ulaze menja na opadajuću ivicu takta, dok check_process poredi na uzlaznu), pre ulaska u glavnu petlju poređenja preskače se fir_ord – 2 dodatnih ciklusa. Pri prijavi neslaganja koristi se severity error (ne failure), čime se simulacija ne prekida na prvi mismatch, već se u konzoli ispisuju sve eventualne razlike (u hex formatu, preko funkcije hstr), radi lakšeg debugovanja. Pokretanjem simulacije u trajanju od 1000 ns potvrđeno je da ne dolazi ni do jednog result mismatch! izveštaja. Proces upisa koeficijenata:



Na slici vidimo da je prvi ulazni odbirak `data_i_s` koji pristiže ima vrednost `0xfc8e24` što je proverom prve linije `expected.txt` (11111111110101001010000) i pretvaranjem u hex: 1111 1111 1110 1010 0101 0000 = FFEA50, dobijemo podatak sa sledeće slike koji se pojavljuje na signal `data_o_s` kao izlazni sa 5 taktova kašnjenja, što odgovara redu filtra (`fir_ord` = 5).

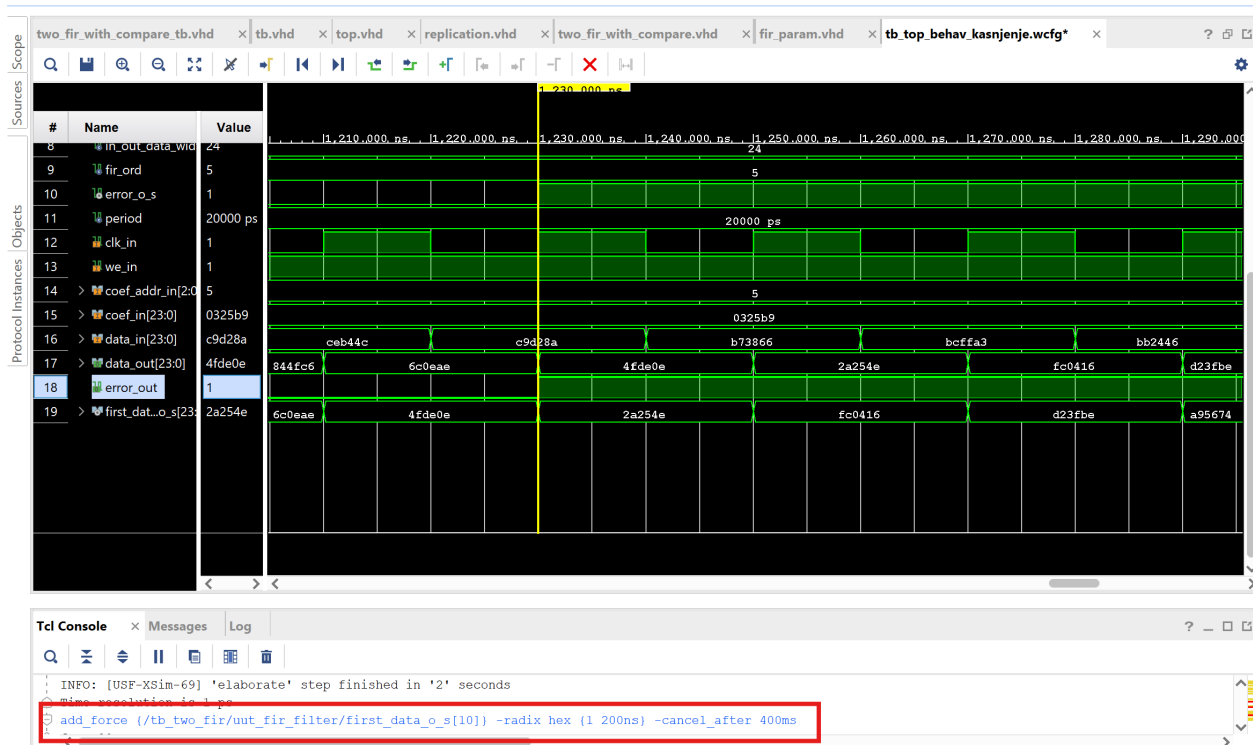


Pokretanjem simulacije potvrđeno je da testbench ispravno prolazi kroz obe faze rada. U prvoj fazi (0–120 ns) signal `we_in` je aktivan dok se redom upisuje 6 koeficijenata filtra (`coef_addr_in` = 0 do 5, u skladu sa `fir_ord` = 5), dok je `data_in` fiksiran na nulu. Nakon završetka upisa, počinje faza filtriranja: `data_in` prima uzastopne odbirke iz `input.txt`, a `data_out` se, nakon kašnjenja od 5 taktova (uvedenog sistoličkim MAC lancem opisanim u poglavlju 2.3), počinje menjati u skladu sa FIR filtriranjem ulaznog niza — vrednosti na izlazu predstavljaju težinsku sumu poslednjih 6 ulaznih odbiraka, a ne kopiju ulaznog signala, što je i očekivano ponašanje FIR filtra. Signal `error_out` ostaje na 0 tokom cele simulacije, potvrđujući da se izlazi dva interna FIR filtra (`first_data_o_s`, `second_data_o_s`) u potpunosti poklapaju u odsustvu veštački unete greške.

Forsiranjem greške:

```
add_force {/tb_two_fir/uut_fir_filter/first_data_o_s[10]} -radix hex {1 200ns} -cancel_after 400ms
```

na `first_data_o_s[10]` (preko `add_force` komande u Tcl konzoli) uočava se da `error_o_s` prelazi na 1, potvrđujući da FD blok ispravno detektuje neslaganje između dva interna FIR filtra na nivou pojedinačne replike, bez ikakve switch/prekidačke logike (ona ne postoji na ovom nivou hijerarhije — dodaje se tek u `replication.vhd`, poglavlje 2.4). Takođe se uočava da `data_out` prati `first_data_o_s` sa jednim taktom dodatnog kašnjenja, usled dodatnog izlaznog registra (`data_out_s`) u `two_fir_with_compare.vhd`.



3.2 Testbench nivoa filtra (fir_param)

Druga verzija cilja samostalni fir_param modul, bez FD/komparatorske logike, i dodaje check_process koji čita golden vrednosti iz expected.txt i poredi ih sa stvarnim izlazom simulacije:

```
if (abs(signed(tmp) - signed(data_o_s)) > "000000000000000000000111") then
```

```
    report "result mismatch!" severity failure;
```

```
end if;
```

Bitna projektantska odluka ovde je da se poređenje **ne** vrši na bit-tačnu jednakost, već sa tolerancijom od ± 7 (binarno 000...0111), na 24-bitnoj skali. Ovo je razumna mera predostrožnosti protiv sitnih razlika u zaokruživanju između MATLAB golden modela i VHDL implementacije (opisanih u poglavlju verifikacije golden vektora), ali istovremeno znači da testbench neće detektovati greške čija je magnituda manja od te tolerancije.

Vredna je pažnje i latencija između stimulusa i provere: proces check_process počinje da čita expected.txt odmah po detekciji start_check = '1', bez eksplicitnog pomeranja za kašnjenje koje uvodi sam FIR filter (opisano u poglavlju 2.3 — svaka MAC sekcija dodaje jedan takt kašnjenja, plus izlazni registar fir_param modula). Ako golden fajl expected.txt nije unapred pomeren za taj

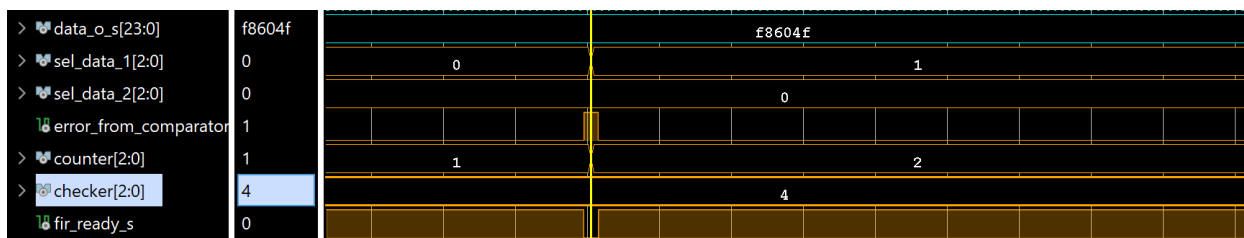
broj odbiraka (ili ako testbench eksplicitno ne uvodi to kašnjenje pre početka poređenja), prvih nekoliko poređenja će se vršiti između odbiraka koji fizički još nije stigao na izlaz i pogrešne golden vrednosti, što može proizvesti lažno javljanje neslaganja na samom početku simulacije.

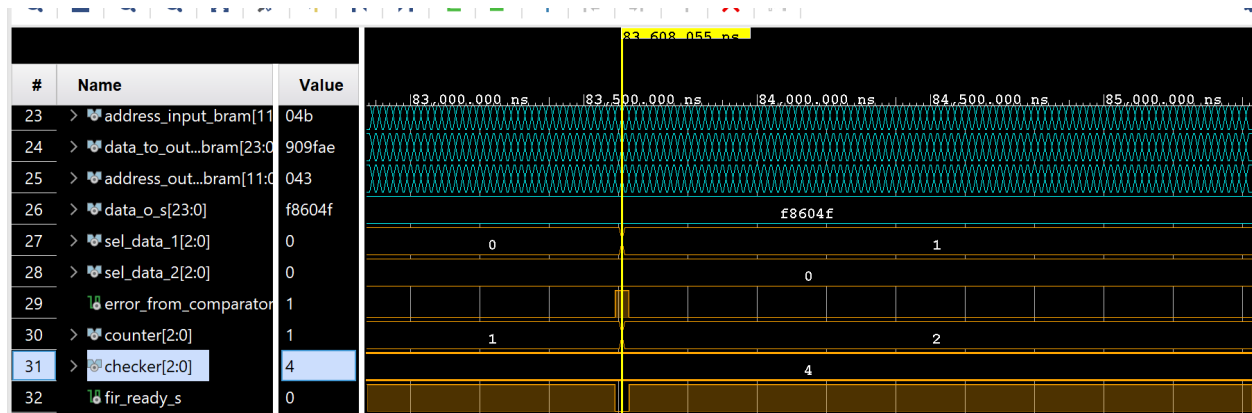
3.3 Testbench nivoa celog sistema (top)

Treća, najkompletnija verzija testira ceo top modul, uključujući ulazni/izlazni BRAM i replication blok. Proces stim_process prati stvarni protokol rada sistema opisan u poglavlju 2.1:

1. Aktivira reset (`rst_i_s <= '1'`), zatim ga oslobađa i postavlja `en_i_s <= '1'`.
2. Upisuje `fir_ord + 1` koeficijenata, isto kao u prethodnim testbench-ovima.
3. Upisuje `RAM_DEPTH (4096)` ulaznih odbiraka direktno u ulazni BRAM preko `data_i_s / addr_write_i_s`, umesto da ih direktno vodi na ulaz filtra — jer u top arhitekturi ulazni podatak prvo prolazi kroz bram (poglavlje 2.1), a tek zatim se automatski čita i prosleđuje u replication blok.
4. Postavlja `start_i_s <= '1'` da pokrene automatsko čitanje iz ulaznog BRAM-a i obradu.
5. Čeka signal `ready_i_s` koji BRAM/top logika generiše po završetku obrade svih odbiraka (opisano u poglavlju 2.1).
6. Redom postavlja adrese čitanja (`addr_read_i_s`) da izvuče sve obrađene rezultate iz izlaznog BRAM-a.

Proces `check_process` ovde poredi izlaz `data_o_s` sa golden fajlom `expected_top.txt` (poseban golden fajl u odnosu na `expected.txt` korišćen u 3.2 — jer prolazi kroz kompletan pair-and-a-spares put, uključujući dodatnu latenciju uvedenu MUX-evima i registrovanim poređenjem u `replication.vhd`), sa istom tolerancijom ± 7 kao u prethodnom testbench-u.





Forcovanjem skripte:

source {C:/Users/Korisnik/Desktop/DSONG/projekat/scripts/force_error_for_top.tcl}

na slici iznad uočavamo da se pri detektovanju pojave greške (error_from_comparator = 1),

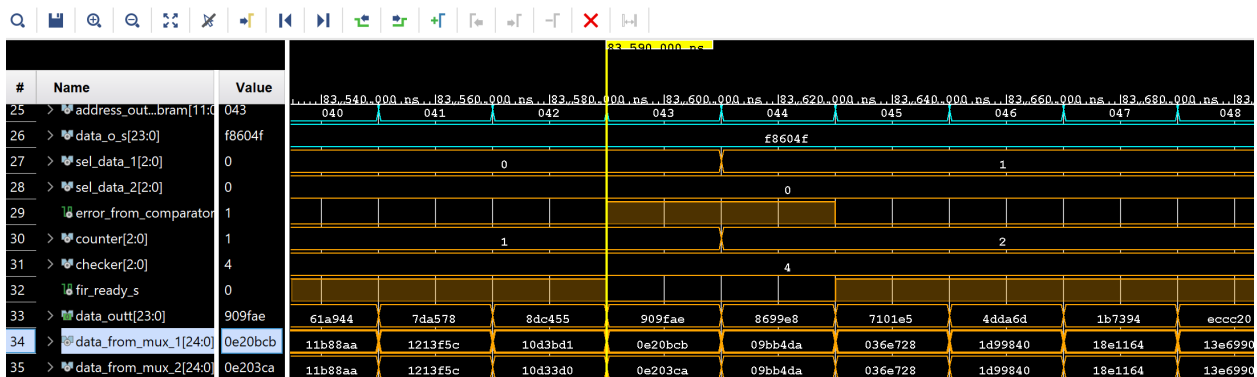
counter se uvećava i selekциони ulaz mux-a menja vrednost i bira da na svoj izlaz prosledi

vrednost koja je na ulazu 1 umesto na ulazu 0. fir_ready_s padne na 0 tačno u trenutku greške

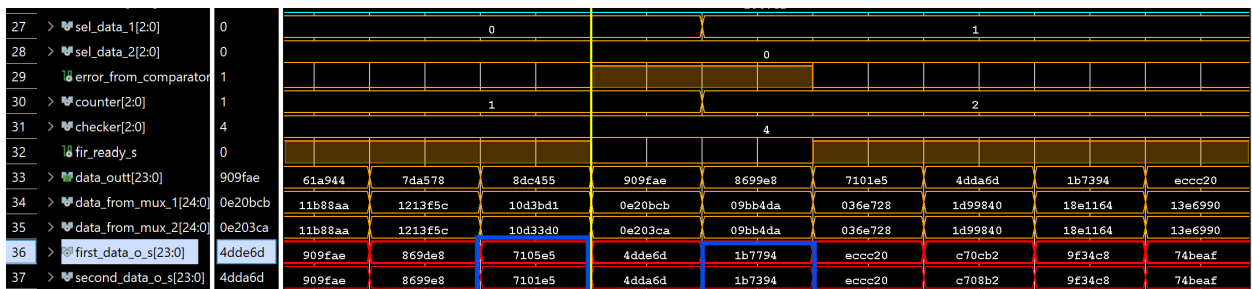
— jer sistem privremeno signalizira da izlaz nije pouzdan dok se ne završi prebacivanje. Checker

predstavlja fiksnu granicu resursa redundanse (number_of_replication - 1), dok counter

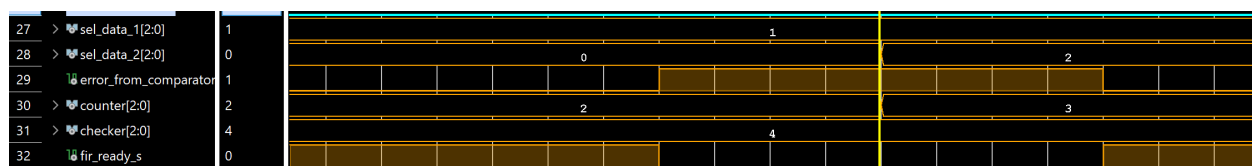
predstavlja trenutnu potrošnju tog resursa — kad se poklope, redundansa je iscrpljena.



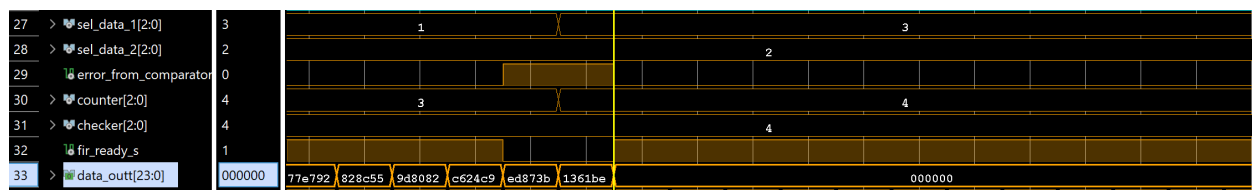
U koloni gde su data_from_mux_1 i data_from_mux_2 različiti, error_from_comparator je 1, a čim ponovo postanu identični, error_from_comparator pada na 0.



Signal	Vrednost pre greske	Vrednost u trenutku greske
first_data_o_s (modul 0)	869de8	7105e5
second_data_o_s (modul 0)	869de8	7101e5
error_s (modul 0)	0	1
data_from_mux_1	1213f5c	10d3bd1
data_from_mux_2	1213f5c	10d33d0
error_from_comparator	0	1
sel_data_1	0	1 (u sledecem taktu)
counter	1	2



sel_data_2 = 2 → gledajući pool za MUX2 ({module1(idx0), module2(idx1), module3(idx2), module4(idx3)}), indeks 2 odgovara modulu 3. Dakle MUX2 se prebacio sa modula 1 (koji je pukao) na modul 3 (sledeća slobodna rezerva). Dokazuje da counter deli "budžet" rezervi između MUX1 i MUX2 zajedno (nema duplog korišćenja iste rezerve): modul 2 je već potrošen od strane MUX1 (sel_data_1=1), pa MUX2 preskače tu vrednost i uzima sledeću slobodnu (counter=2 → modul 3).



Kada svi rezervni moduli budu istrošeni, data_outt se postavlja na nulu, kao što je u dokumentaciji prethodno već bilo opisano.

Pokretanjem skripte za remove force-ova zaključila sam da trenutna implementacija ne sadrži mehanizam za resetovanje detektovane greške (signal rst_i je proveden kroz hijerarhiju, ali nije iskorišćen u replication.vhd), što znači da je detekcija greške permanentna do sledećeg power-on reseta cele simulacije/sistema. Ovo predstavlja potencijalno proširenje sistema — dodavanje

rst_i logike koja bi čistila error_s, sel_data_1/2 i counter signale, omogućavajući oporavak nakon zamene neispravnog modula.

Ovo je realizovano pokretanjem komande:

```
source {C:/Users/Korisnik/Desktop/DSONG/projekat/scripts/remove_force_for_top.tcl}
```

4.1 Analiza utrošenosti DSP resursa i frekvencije rada

Broj korišćenih DSP48 blokova raste linearno sa brojem redundantnih modula (36 DSP-ova po modulu), sve dok se ne dostigne fizička granica dostupnih resursa na ciljanom FPGA-u (220 DSP48 blokova). Kod konfiguracije sa 7 modula, teorijska potreba (252 DSP-a) premašuje raspoloživ broj, usled čega Vivado alat automatski implementira deo množača u LUT logici umesto u dedikiranim DSP blokovima, što je vidljivo kroz nesrazmerno veliko povećanje LUT resursa (sa 392 na 1958). Ovo predstavlja praktičnu gornju granicu skalabilnosti predložene arhitekture na datoj FPGA platformi, i ukazuje da bi za veći stepen redundanse bila potrebna pločica sa više DSP48 resursa.

number_of_replication	DSP48 (Used)	DSP %	LUT	WNS (ns)	f_max (MHz)
3	108	49.09%	279	1.310	≈68.07
5	180	81.82%	392	0.120	≈62.97
7	216	98.18%	1958	0.099	≈62.86

Napomena: Za konfiguraciju sa 7 modula, teorijska potreba za DSP resursima ($7 \times 36 = 252$) premašuje raspoloživ broj DSP48 blokova na ciljanom FPGA-u (220), usled čega je deo množača implementiran u LUT logici, što je vidljivo kroz nesrazmerno povećanje broja korišćenih LUT-ova.

Sinteza i implementacija izvršene su uz zadat vremenski ograničavajući uslov (constraint) od 16 ns (create_clock -period 16), što odgovara ciljanom radnoj frekvenciji od 62.5 MHz. Za sve tri testirane konfiguracije (3, 5 i 7 modula) dobijen je pozitivan WNS, što znači da su sve tri konfiguracije uspešno zadovoljile taj zahtev, sa stvarnom maksimalnom radnom frekvencijom (f_max) koja opada sa 68.07 MHz na 62.86 MHz kako raste broj redundantnih modula — usled dodatne kombinacije logike u MUX/komparatorskoj mreži replication.vhd modula (poglavlje 2.4), koja produžava kritičnu putanju kola. Ipak, i pri najvećem testiranom broju modula (7), margina (slack) ostaje pozitivna (WNS = 0.099 ns), što potvrđuje da arhitektura zadržava

zahtevanu radnu frekvenciju uprkos povećanju broja replika, sve do granice DSP resursa opisane iznad.

number_of_replication=3:

Graph Table			
Resource	Estimation	Available	Utilization %
LUT	279	53200	0.52
FF	307	106400	0.29
BRAM	6	140	4.29
DSP	108	220	49.09
IO	104	125	83.20
BUFG	1	32	3.13

Design Timing Summary			
Setup	Hold	Pulse Width	
Worst Negative Slack (WNS): 1.310 ns	Worst Hold Slack (WHS): 0.032 ns	Worst Pulse Width Slack (WPWS): 7.500 ns	
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	
Total Number of Endpoints: 5784	Total Number of Endpoints: 5784	Total Number of Endpoints: 428	
All user specified timing constraints are met.			

f_max = 1000 / (16 - 1.310) = 1000 / 14.69 ≈ 68.07 MHz

number_of_replication=5:

Utilization

Post-Synthesis | Post-Implementation

Graph | Table

Resource	Utilization	Available	Utilizatio...
LUT	392	53200	0.74
FF	456	106400	0.43
BRAM	6	140	4.29
DSP	180	220	81.82
IO	104	125	83.20
BUFG	1	32	3.13

Power

Summary | On-Chip

Total On-Chip Power:

0.26 W

Junction Temperature:

28.0 °C

Thermal Margin:

57.0 °C (4.8 W)

Effective θ_{JA} :

11.5 °C/W

Power supplied to off-chip devices:

0 W

Confidence level:

Low

Implemented Power Report

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 0.120 ns	Worst Hold Slack (WHS): 0.034 ns	Worst Pulse Width Slack (WPWS): 7.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 9488	Total Number of Endpoints: 9488	Total Number of Endpoints: 649
All user specified timing constraints are met.		

f_max = 1000 / (16 - 0.120) = 1000 / 15.88 ≈ 62.97 MHz

number_of_replication=7:

Utilization

Post-Synthesis | Post-Implementation

Graph | Table

Resource	Utilization	Available	Utilizatio...
LUT	1958	53200	3.68
FF	2180	106400	2.05
BRAM	6	140	4.29
DSP	216	220	98.18
IO	104	125	83.20
BUFG	1	32	3.13

Power

Summary | On-Chip

Total On-Chip Power:

0.331 W

Junction Temperature:

28.8 °C

Thermal Margin:

56.2 °C (4.7 W)

Effective θ_{JA} :

11.5 °C/W

Power supplied to off-chip devices:

0 W

Confidence level:

Low

Implemented Power Report

ports	Design Runs	DRC	Methodology	Power	Timing	x
Design Timing Summary						
Setup		Hold		Pulse Width		
Worst Negative Slack (WNS): 0.099 ns		Worst Hold Slack (WHS): 0.079 ns		Worst Pulse Width Slack (WPWS): 7.500 ns		
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns		
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		
Total Number of Endpoints: 11596		Total Number of Endpoints: 11596		Total Number of Endpoints: 2409		
All user specified timing constraints are met.						

$f_{\text{max}} = 1000 / (16 - 0.099) \approx 62.9 \text{ MHz}$